

2^ο ΙΕΚ ΗΡΑΚΛΕΙΟΥ

ΜΑΘΗΜΑ: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ (C++)

Φύλλο Ανάθεσης Εργασίας Αριθμ. 16

Υπερφόρτωση

Δραστηριότητα 1 – Υπερφόρτωση (Overloading) μεθόδων

Η C++ υποστηρίζει υπερφόρτωση συναρτήσεων και τελεστών. Μπορούμε κατά συνέπεια να ορίσουμε περισσότερες από μια συναρτήσεις με το ίδιο όνομα στην ίδια εμβέλεια, που θα έχουν όμως διαφορετικά ορίσματα. Όταν καλείται μια υπερφορτωμένη συνάρτηση ο μεταγλωττιστής ορίζει ποια συνάρτηση θα καλέσει, κρίνοντας από τα ορίσματα που έχουν χρησιμοποιηθεί στην κλήση της συνάρτησης. Το ίδιο συμβαίνει και με τις μεθόδους μιας κλάσης. Στη σελίδα https://www.tutorialspoint.com/cplusplus/cpp_overloading.htm θα βρείτε ένα παράδειγμα όπου δηλώνονται τρεις μέθοδοι με το ίδιο όνομα (print) στην ίδια κλάση.

Δραστηριότητα 2 – Υπερφόρτωση κατασκευαστών

Δημιουργήστε ένα νέο φάκελο με όνομα FE15 όπου θα αποθηκεύσετε τα παραδείγματα του Φύλλου Εργασίας. Δημιουργήστε ένα αρχείο με όνομα overloaded_constructors.cpp με το παρακάτω περιεχόμενο:

```
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;
public:
    Rectangle ();
    Rectangle (int,int);
    int getWidth() {return width;}
    int getHeight() {return height;}
    int area() {return width*height;}
};

Rectangle::Rectangle () {
    width = 5;
    height = 5;
}

Rectangle::Rectangle (int a, int b) {
    width = a;
    height = b;
}

int main () {
    Rectangle recta (3,4);
    Rectangle rectb;
    cout << "RectA width = " << recta.getWidth() << endl;
    cout << "RectA height = " << recta.getHeight() << endl;
    cout << "RectA area: " << recta.area() << endl << endl;
    cout << "RectB width = " << rectb.getWidth() << endl;
    cout << "RectB height = " << rectb.getHeight() << endl;
    cout << "RectB area: " << rectb.area() << endl;

    return 0;
}
```

Στο παραπάνω παράδειγμα ορίσαμε δύο κατασκευαστές:

- τον κατασκευαστή χωρίς ορίσματα, εξ' ορισμού κατασκευαστή (default constructor), που καλείται όταν δημιουργούμε ένα αντικείμενο τύπου Rectangle χωρίς να ορίσουμε τις ιδιότητές του, και
- τον κατασκευαστή με δύο ορίσματα, που καλείται όταν δημιουργούμε ένα αντικείμενο τύπου Rectangle με συγκεκριμένο πλάτος και ύψος

Για να καλέσουμε τον εξ' ορισμού κατασκευαστή δε βάζουμε παρενθέσεις(). Αν δηλώναμε, για παράδειγμα, **Rectangle rectc();** ο compiler θα νόμιζε ότι θέλουμε να καλέσουμε τη συνάρτηση rectc() που επιστρέφει Rectangle.

Δραστηριότητα 3 – Ορισμός ιδιοτήτων στους κατασκευαστές

Οι κατασκευαστές χρησιμοποιούνται συχνά για να προσδιορίσουμε αρχικές τιμές στις ιδιότητες του αντικειμένου που δημιουργούμε. Σε αυτή την περίπτωση μπορούμε να αρχικοποιήσουμε τις ιδιότητες αυτές πριν το κυρίως σώμα του κατασκευαστή βάζοντας άνω-κάτω τελεία(:) και μετά τη λίστα με τις αρχικοποιήσεις. Στο προηγούμενο παράδειγμα ορίσαμε τον κατασκευαστή με παραμέτρους, ως εξής::

```
Rectangle::Rectangle (int x, int y) { width=x; height=y; }
```

Εναλλακτικά θα μπορούσαμε, όπως παρακάτω, να αρχικοποιήσουμε την ιδιότητα width σύμφωνα με την παράμετρο x, πριν το κυρίως σώμα της μεθόδου:

```
Rectangle::Rectangle (int x, int y) : width(x) { height=y; }
```

ή ακόμα και:

```
Rectangle::Rectangle (int x, int y) : width(x), height(y) { }
```

Δημιουργήστε ένα αρχείο με όνομα complex_constructor.cpp με το παρακάτω περιεχόμενο και εκτελέστε το:

```
// member initialization
#include <iostream>
using namespace std;

class Circle {
    double radius;
public:
    Circle(double r) : radius(r) { }
    double area() {return radius*radius*3.14159265;}
};

class Cylinder {
    Circle base;
    double height;
public:
    Cylinder(double r, double h) : base (r), height(h) {}
    double volume() {return base.area() * height;}
};

int main () {
    Cylinder foo (10,20);

    cout << "foo's volume: " << foo.volume() << '\n';
    return 0;
}
```

Τι πιστεύετε ότι κάνει η εντολή base(r) στον κατασκευαστή της κλάσης Cylinder;